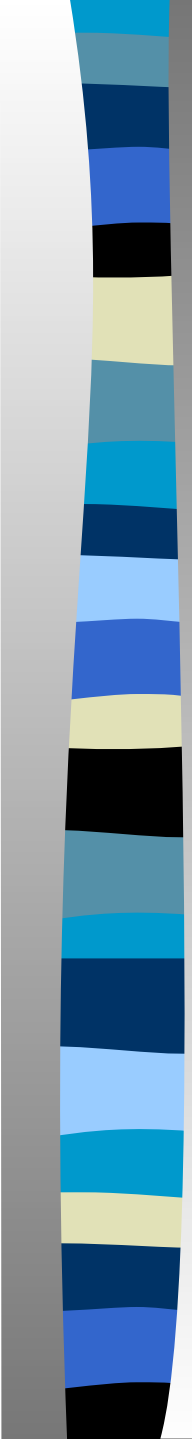


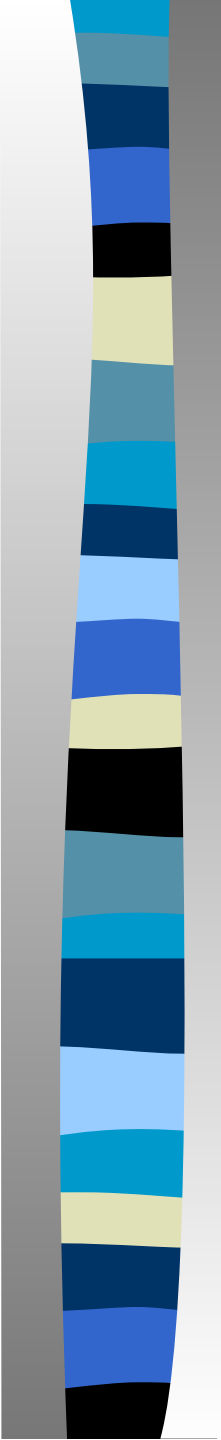
UNIT 2

ARRAYS

- 
- An array is a finite, ordered and collection of homogeneous data elements.
 - Array is finite because it contains only limited no of elements
 - It is ordered as all the elements are stored one by one in contiguous locations of computer memory in a linear ordered fashion.
 - All the elements of an array are of the same data type only and hence it is termed as collection of homogeneous elements.
 - An array is known as linear data structure because all elements of the array are stored in a linear order.
 - It is generally a built-in data type in all programming languages.

Terminologies

- Type-kind of data type
- Base-memory address location where 1st element in a array is located
- Index-All elements can be referenced by a subscript known as Index
- Range of Index-Boundaries –Lower Bound and Upper Bound



Arrays

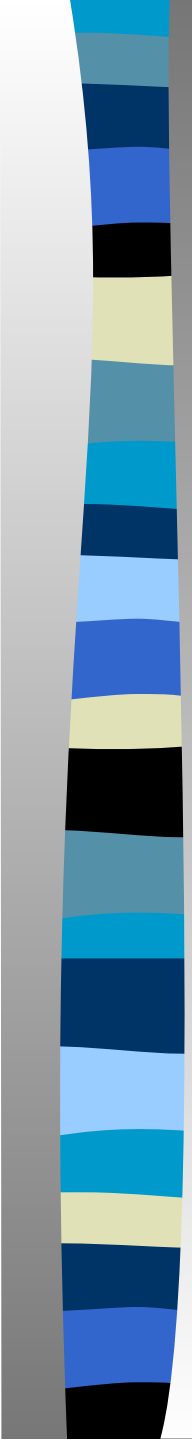
Array: a set of **index** and **value**

data structure

For each index, there is a value associated with that index.

representation (possible)

implemented by using consecutive memory.

- 
- A **fixed array** - size or length is determined when the array is created and/or allocated. [\[1\]](#)
 - A **dynamic array** is a random access, variable-size list data structure that allows elements to be added or removed.
 - It is supplied with standard libraries in many modern programming languages.
 - Dynamic arrays overcome a limit of static arrays, which have a fixed capacity that needs to be specified at allocation. [\[2\]](#)
 -

Structure *Array* is

objects: A set of pairs $\langle index, value \rangle$ where for each value of *index* there is a value from the set *item*. *Index* is a finite ordered set of one or more dimensions, for example, $\{0, \dots, n-1\}$ for one dimension, $\{(0,0),(0,1),(0,2),(1,0),(1,1),(1,2),(2,0),(2,1),(2,2)\}$ for two dimensions, etc.

Functions:

for all $A \in \text{Array}$, $i \in \text{index}$, $x \in \text{item}$, $j, \text{size} \in \text{integer}$

Array Create(j, list) ::= **return** an array of *j dimensions* where *list* is a *j-tuple* whose *ith element* is the *size* of the *ith dimension*. *Items* are undefined.

Item Retrieve(A, i) ::= **if** ($i \in \text{index}$) **return** the item associated with index value *i* in array *A*
else return error

Array Store(A, i, x) ::= **if** ($i \in \text{index}$)
return an array that is identical to array *A* except the new pair $\langle i, x \rangle$ has been inserted **else return error**

end array

*Structure 2.1: Abstract Data Type *Array* (p.50)

Arrays in C

`int list[5], *plist[5];`

`list[5]`: five integers

`list[0]`, `list[1]`, `list[2]`, `list[3]`, `list[4]`

`*plist[5]`: five pointers to integers

`plist[0]`, `plist[1]`, `plist[2]`, `plist[3]`, `plist[4]`

implementation of 1-D array

- Compiler allocates 5 consecutive memory locations for the above declaration
- Each memory location is large enough to hold a single integer
- Address of the first element `list[0]` is called the **base address**
- If the size of an integer on your machine is denoted by *sizeof(int)* then memory address can be written as $\alpha + i * \text{sizeof}(\text{int})$

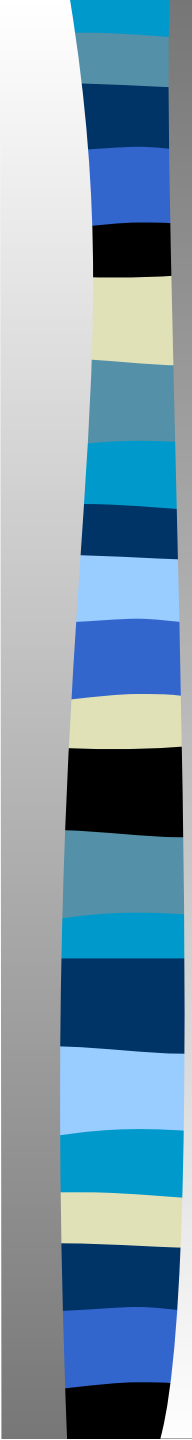
`list[0]` base address = α

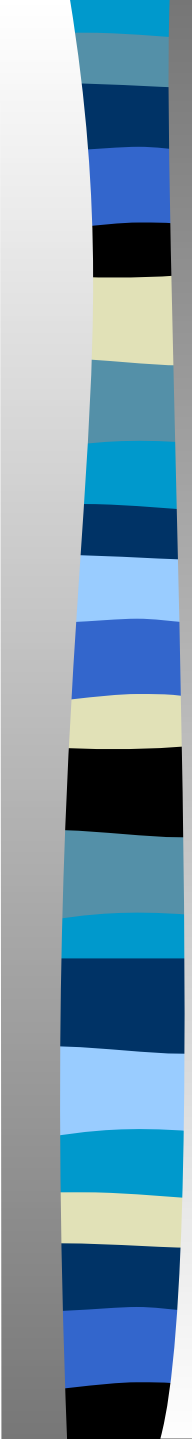
`list[1]` $\alpha + 1 * \text{sizeof}(\text{int})$

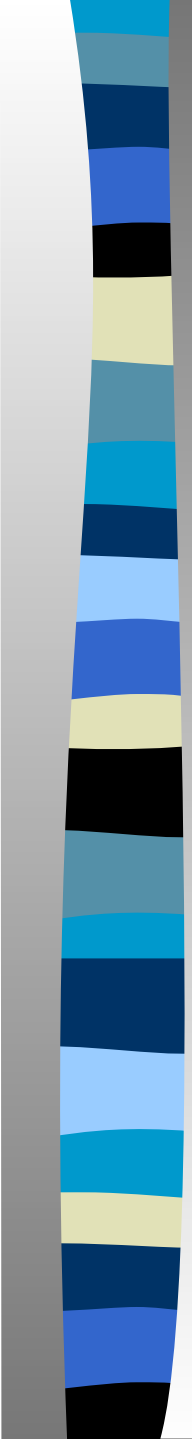
`list[2]` $\alpha + 2 * \text{sizeof}(\text{int})$

`list[3]` $\alpha + 3 * \text{sizeof}(\text{int})$

`list[4]` $\alpha + 4 * \text{sizeof}(\text{int})$

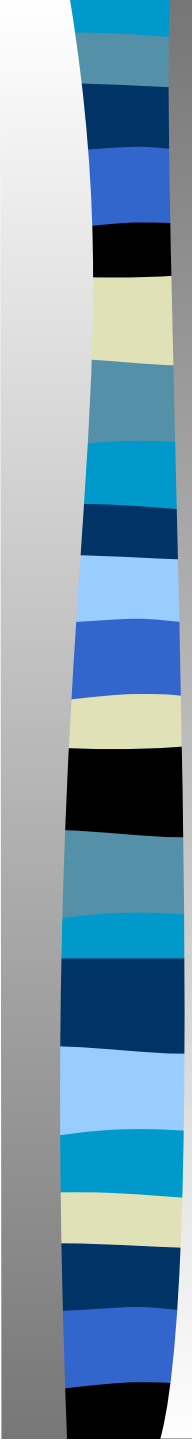
- 
- Compare `int *list1` and `int list2[5]` in C.
 - Same : list1 and list2 are **pointers to an int**.
 - Difference : list2 reserves **five memory locations**.
 - list2 is a pointer to list2[0] and list2+i is a pointer to list2[i]
 - In C we do not multiply the offset i with the size of the type to get to the appropriate element of the array
 - Thus, regardless of the type of the array list2. it is always the case that (list2+i) equals &list2[i]
 - So *(list2+i) equals list2[i]

- 
- Assume that we have the following declaration
`int one[] = {0, 1, 2, 3, 4};`
Goal: print out address and value
 - For ex : Write a function that prints out both the address of the *i*th element of this array and the value found at this address
 - The program `print1` uses pointer arithmetic
 - The function is invoked as `print1(&one[0],5)`
 - The address of the *i*th element is simply `ptr+i`
 - To obtain the value of the *i*th element we use the dereferencing operator, `*`.
 - Thus, `*(ptr+i)` indicates that we want the contents of the `ptr+i` position rather than the address



Example: 1-dimension array addressing

```
void print1(int *ptr, int rows)
{
/* print out a one-dimensional array using a pointer */
    int i;
    printf("Address Contents\n");
    for (i=0; i < rows; i++)
        printf("%08u%05d\n", ptr+i, *(ptr+i));
    printf("\n");
}
```



call print1(&one[0], 5)

Address	Contents
1228	0
1230	1
1232	2
1234	3
1236	4

***Figure 2.1: One- dimensional array addressing (p.53)**

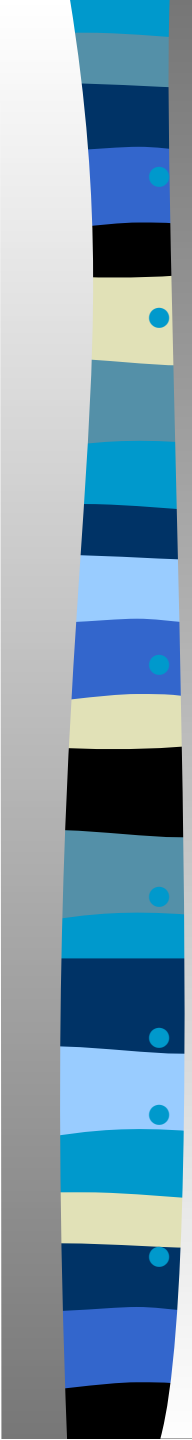
POLYNOMIALS

- **Arrays** are not only data structures in their own right, it can also be used to implement other abstract data types.
- For instance, let us consider one of the simplest and most commonly found data structures: *the ordered, or linear, list*.
- Examples include
 - Days of the week: (Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday)
 - Values in a deck of cards: (Ace, 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, King)
 - Floors of a building: (basement, lobby, mezzanine, first, second)
 - Years the United States fought in World War II: (1941, 1942, 1943, 1944, 1945)
 - Years Switzerland fought in World War II: ()
- Notice that the years Switzerland fought in World War II is different because it contains no items.
- It is an example of an empty list, which is denoted as ().
- The other lists all contain items that are written in the form (*item*₀, *item*₁, ..., *item*_{*n*-1}).

- Operations on lists include:

- Finding the length, n , of a list.
- Reading the items in a list from left to right (or right to left).
- Retrieving the i th item from a list, $0 \leq i < n$.
- Replacing the item in the i th position of a list, $0 \leq i < n$.
- Inserting a new item in the i th position of a list, $0 \leq i \leq n$. The items previously numbered $i, i+1, \dots, n-1$ become items numbered $i+1, i+2, \dots, n$.
- Deleting an item from the i th position of a list, $0 \leq i < n$. The items numbered $i+1, \dots, n-1$ become items numbered $i, i+1, \dots, n-2$.

- Rather than stating the formal specification of the *ADT list*, it can be explored briefly to its *implementation*.
- The most common implementation is to represent an ordered list as an *array* where the list element, *item_i*, associates with the array index *i*.
- This method is called *sequential mapping* because, assuming the standard implementation of an array, we are storing *item_i*, *item_i + 1* into consecutive slots *i* and *i + 1* of the array.
- Sequential mapping works well for most of the operations listed above.
- Thus, we can *retrieve* an item, *replace* an item, or find the *length* of a list, in constant time.
- We also can read the items in the list, from either direction, by simply changing subscripts in a controlled way.
- Only *insertion and deletion pose problems* since the sequential allocation forces us to move items so that the sequential mapping is preserved.
- It is precisely this overhead that leads us to consider *nonsequential mappings of ordered lists*.



Therefore, it makes sense to look at the problem and see why arrays offer only a partially adequate solution.

- The problem calls for building a set of functions that allow for the manipulation of symbolic polynomials.

Viewed from a mathematical perspective, a polynomial is a sum of terms, where each term has a form ax^e where x is the variable, a is the coefficient, and e is the exponent.

- Two example polynomials are

$$A(x) = 3x^{20} + 2x^5 + 4 \text{ and } B(x) = x^4 + 10x^3 + 3x^2 + 1$$

- The largest (or leading) exponent of a polynomial is called its *degree*.
- Coefficients that are zero are not displayed.
- The term with exponent equal to zero does not show the variable since x raised to a power of zero is 1.
- There are standard mathematical definitions for the sum and product of polynomials.

Polynomial Represent

$\sum a_i x^i$ and $B(x) = \sum b_i x^i$ then:

- Assume that we have two polynomials, $A(x) = \sum a_i x^i$ and $B(x) = \sum b_i x^i$.
$$A(x) + B(x) = \sum (a_i + b_i) x^i$$
- Similarly, we can define **subtraction and division** on polynomials, as well as many other operations.
$$A(x) \cdot B(x) = \sum (a_i x^i \cdot \sum (b_j x^j))$$
- We begin with an **ADT** definition of a polynomial.
- The particular operations in part are a reflection of what will be needed in our subsequent programs to manipulate polynomials.
- We are now ready to make some **representation** decisions.
- A very reasonable first decision requires **unique** exponents arranged in **decreasing order**.
- This requirement considerably **simplifies** many of the **operations**.
- Using this specification and stipulation, we can write a version of **Add** that is closer to a C function (Program 2.4), but is still representation-independent.

structure *Polynomial* is

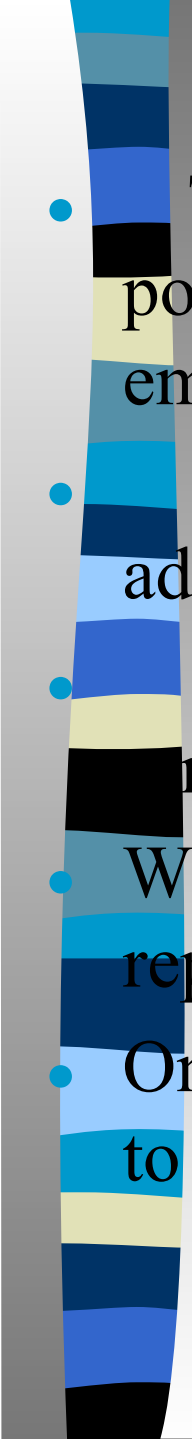
objects: $p(x) = a_1x^{e_1} + \dots + a_nx^{e_n}$; a set of ordered pairs of $\langle e_i, a_i \rangle$ where a_i in *Coefficients* and e_i in *Exponents*, e_i are integers ≥ 0

functions:

for all $poly, poly1, poly2 \in Polynomial$, $coef \in Coefficients$, $expon \in Exponents$

<i>Polynomial</i> Zero()	::=	return the polynomial, $p(x) = 0$
<i>Boolean</i> IsZero(<i>poly</i>)	::=	if (<i>poly</i>) return FALSE else return TRUE
<i>Coefficient</i> Coef(<i>poly</i> , <i>expon</i>)	::=	if (<i>expon</i> $\in$ <i>poly</i>) return its coefficient else return zero
<i>Exponent</i> Lead_Exp(<i>poly</i>)	::=	return the largest exponent in <i>poly</i>
<i>Polynomial</i> Attach(<i>poly</i> , <i>coef</i> , <i>expon</i>)	::=	if (<i>expon</i> $\in$ <i>poly</i>) return error else return the polynomial <i>poly</i> with the term $\langle coef, expon \rangle$ inserted
<i>Polynomial</i> Remove(<i>poly</i> , <i>expon</i>)	::=	if (<i>expon</i> $\in$ <i>poly</i>) return the polynomial <i>poly</i> with the term whose exponent is <i>expon</i> deleted else return error
<i>Polynomial</i> SingleMult(<i>poly</i> , <i>coef</i> , <i>expon</i>)	::=	return the polynomial $poly \cdot coef \cdot x^{expon}$
<i>Polynomial</i> Add(<i>poly1</i> , <i>poly2</i>)	::=	return the polynomial $poly1 + poly2$
<i>Polynomial</i> Mult(<i>poly1</i> , <i>poly2</i>)	::=	return the polynomial $poly1 \cdot poly2$

end *Polynomial*

- 
- This algorithm works by comparing terms from the two polynomials until one or both of the polynomials becomes empty.
 - The **switch** statement performs the comparisons and adds the proper term to the new polynomial, d .
 - If one of the polynomials becomes empty, we copy the remaining terms from the nonempty polynomial into d .
 - With these insights, suppose we now consider the representation question more carefully.
 - One way to represent polynomials in C is to use **typedef** to create the type *poly nomial* as below:

Polynomial Addition

data structure 1:

```
#define MAX_DEGREE 101
```

```
typedef struct {
```

```
    int degree;
```

```
    float coef[MAX_DEGREE];
```

```
} polynomial;
```

```
/* d = a + b, where a, b, and d are polynomials */
```

```
d = Zero( )
```

```
while (! IsZero(a) && ! IsZero(b)) do {
```

```
    switch COMPARE (Lead_Exp(a), Lead_Exp(b)) {
```

```
        case -1: d =
```

```
            Attach(d, Coef (b, Lead_Exp(b)), Lead_Exp(b));
```

```
            b = Remove(b, Lead_Exp(b));
```

```
            break;
```

```
        case 0: sum = Coef (a, Lead_Exp (a)) + Coef ( b, Lead_Exp(b));
```

```
        if (sum) {
```

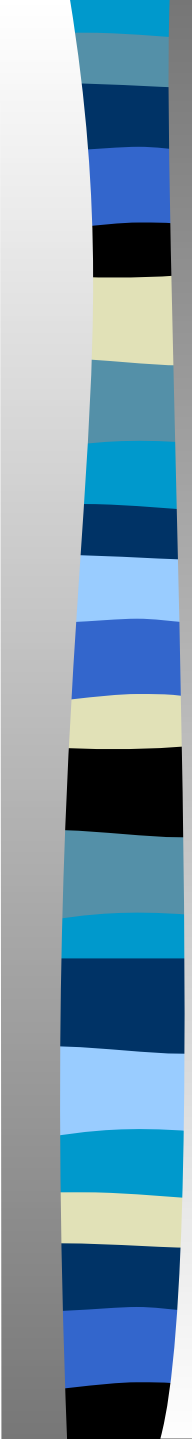
```
            Attach (d, sum, Lead_Exp(a));
```

```
            a = Remove(a , Lead_Exp(a));
```

```
            b = Remove(b , Lead_Exp(b));
```

```
        }
```

```
        break;
```



case 1: d =

```
    Attach(d, Coef (a, Lead_Exp(a)), Lead_Exp(a));
```

```
    a = Remove(a, Lead_Exp(a));
```

```
    }
```

```
}
```

insert any remaining terms of a or b into d

advantage: easy implementation

disadvantage: waste space when sparse

*Program 2.4 :Initial version of *padd* function(p.62)

`a.coef[i] = an-i, 0 ≤ i ≤ n`

- In this representation, coefficients are stored in the order of decreasing exponents, such that `a.coef[i]` is the coefficient of x (to the power of) $n-i$ provided a term with exponent $n-i$ exists; otherwise, $a.coef[i] = 0$.
- Although this representation leads to very simple algorithms for most of the operations, it wastes a **lot of space**.
- For instance, if $a.degree \ll MAX-DEGREE$, then most of the positions in `a.coef[MAX-DEGREE]` is *not needed*.
- The same argument applies if the polynomial is **sparse**, that is, the number of terms with **nonzero coefficient** is small relative to the **degree** of the polynomial.
- To preserve space, an alternate representation is devised which uses only one **global array**, *terms*, to store all polynomials.
- The C declarations needed are:

- 
- The index of the first term of A and B is given by *starta* and *startb*, respectively, while *finisha* and *finishb* give the index of the last term of A and B .
 - The index of the next free location in the array is given by *avail*.
starta = 0, *finisha* = 1, *startb* = 2, *finishb* = 5, and *avail* = 6.
 - This representation **does not impose any limit on the number of polynomials** which is placed in the *terms*.
 - The only stipulation is that the **total number of nonzero terms** must be **no more than MAX-TERMS**.
 - It is worth pointing out the difference between our specification and our representation.
 - Specification used *poly* to refer to a polynomial, and representation translated *poly* into a *<start, finish>* pair.
 - Therefore, to use $A(x)$ we must pass in *starta* and *finisha*.
 - Any polynomial A that has n nonzero terms has *starta* and *finisha* such that $finisha = starta + n - 1$.

Data structure 2: use one global array to store all polynomials

$$A(X)=2X^{1000}+1$$

$$B(X)=X^4+10X^3+3X^2+1$$

***Figure 2.2:** Array representation of two polynomials

(p.63)

	<i>starta</i>	<i>finisha</i>	<i>startb</i>			<i>finishb</i>	<i>avail</i>
	↓	↓	↓			↓	↓
<i>coef</i>	2	1	1	10	3	1	
	1000	0	4	3	2	0	
<i>exp</i>	0	1	2	3	4	5	6

specification

poly

A

B

representation

<start, finish>

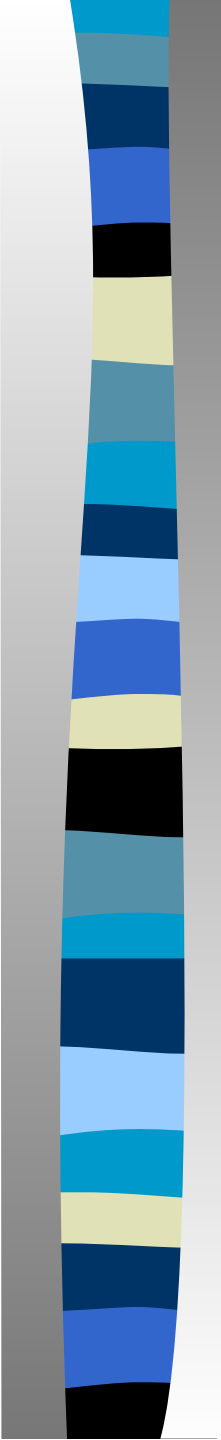
<0,1>

<2,5>

This representation certainly solves the problem of many zero terms

since $A(x) = 2x \text{ to the power } 1000 + 1$ uses only **six units** of storage one for *starta*, one for *finisha*, **two** for the **coefficients**, and **two** for the **exponents**.

- However, when all the terms are **nonzero**, the current representation requires about **twice** as much space as the first one.
- Unless we know before hand that each of our polynomials has **few zero terms**, this current representation is probably **better**.
- C function that adds two polynomials, A and B , represented as above to obtain $D = A + B$.
- To produce $D(x)$, *padd* (Program 2.5) adds $A(x)$ and $B(x)$ term by term.
- Starting at position *avail*, *attach* (Program 2.6) places the terms of D into the array, *terms*.
- If there is **not enough space** in *terms* to accommodate D , an **error message** is printed to the standard error device and exit the program with an error condition.



storage requirements: start, finish, $2 * (\text{finish} - \text{start} + 1)$

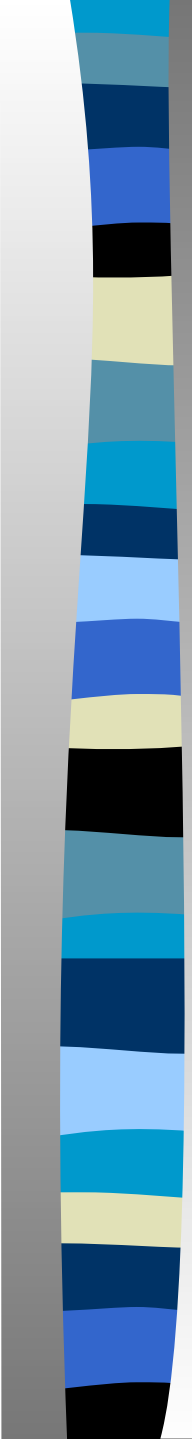
nonsparse: twice as much as (1)
 when all the items are nonzero

```
MAX_TERMS 100 /* size of terms array */
```

```
typedef struct {  
    float coef;  
    int expon;  
} polynomial;
```

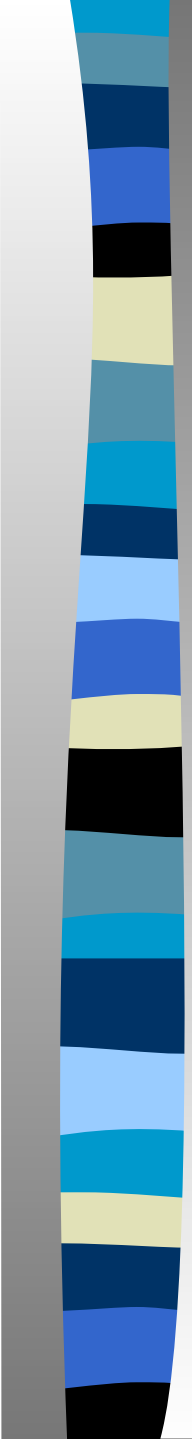
```
polynomial terms[MAX_TERMS];
```

```
int avail = 0;  
*(p.62)
```

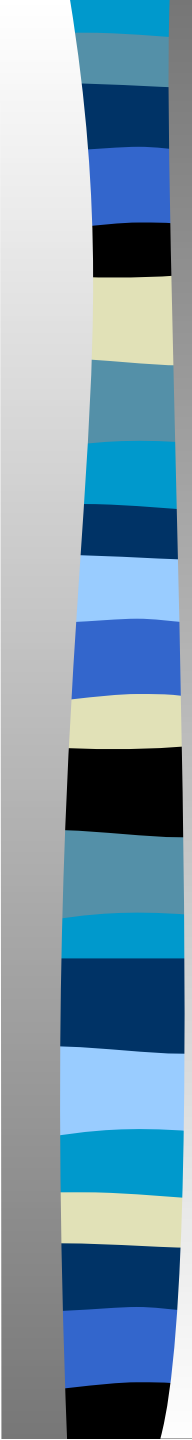


Add two polynomials: $D = A + B$

```
void padd (int starta, int finisha, int startb, int finishb,  
           int * startd, int * finishd)  
{  
    /* add A(x) and B(x) to obtain D(x) */  
    float coefficient;  
    *startd = avail;  
    while (starta <= finisha && startb <= finishb)  
        switch (COMPARE(terms[starta].expon,  
                        terms[startb].expon)) {  
        case -1: /* a expon < b expon */  
            attach(terms[startb].coef, terms[startb].expon);  
            startb++;  
            break;
```



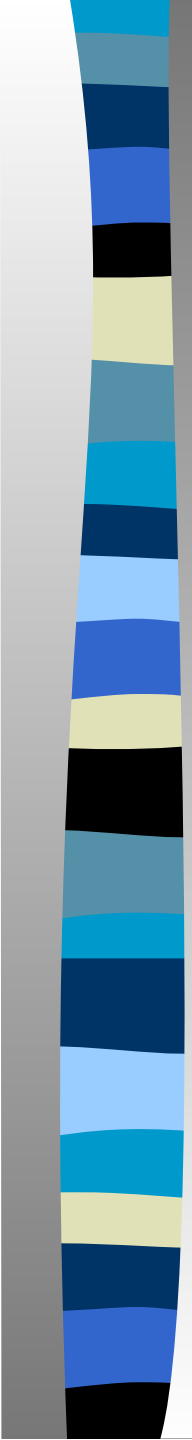
```
case 0: /* equal exponents */
    coefficient = terms[starta].coef +
                terms[startb].coef;
    if (coefficient)
        attach (coefficient, terms[starta].expon);
    starta++;
    startb++;
    break;
case 1: /* a expon > b expon */
    attach(terms[starta].coef, terms[starta].expon);
    starta++;
}
```



```
/* add in remaining terms of A(x) */  
for( ; starta <= finisha; starta++)  
    attach(terms[starta].coef, terms[starta].expon);  
/* add in remaining terms of B(x) */  
for( ; startb <= finishb; startb++)  
    attach(terms[startb].coef, terms[startb].expon);  
*finishd = avail - 1;  
}
```

Analysis: $O(n+m)$ where n (m) is the number of nonzeros in A (B).

***Program 2.5: Function to add two polynomial (p.64)**



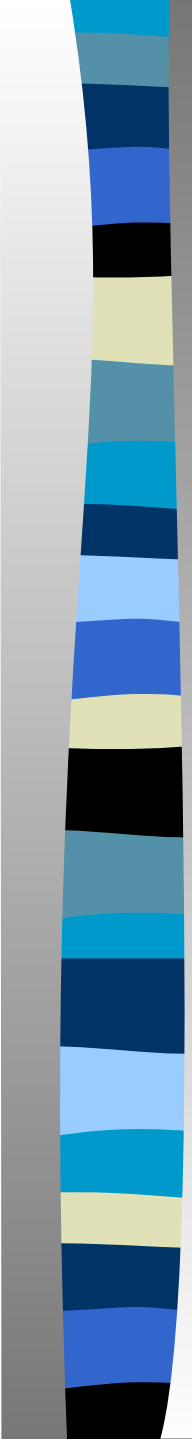
```
void attach(float coefficient, int exponent)
{
/* add a new term to the polynomial */
if (avail >= MAX_TERMS) {
    fprintf(stderr, "Too many terms in the polynomial\n");
    exit(1);
}
terms[avail].coef = coefficient;
terms[avail++].expon = exponent;
}
```

***Program 2.6: Function to add a new term (p.65)**

Problem: Compaction is required
when polynomials that are no longer needed.
(data movement takes time.)

- **Analysis of *padd***-. The number of nonzero terms in A and in B are the most **important factors** in the time complexity.
- Therefore, let tn and n be the number of nonzero terms in A and B , respectively.
- If $tn > 0$ and $n > 0$, the **while** loop is **entered**.
- Each iteration of the loop requires **$O(1)$ time**.
- At each iteration, **increment** the value of $starta$ or $startb$ or both.
- Since the iteration **terminates** when either $starta$ or $startb$ exceeds $finisha$ or $finishb$, respectively, the number of iterations is bounded by $m + n - 1$.
- This worst case occurs when:

$$A(x) = \sum_{i=0}^n x^{2i} \text{ and } B(x) = \sum_{i=0}^n x^{2i+1}$$

- 
- The **time** for the remaining two loops is bounded by $O(n + m)$ because we **cannot iterate** the **first loop** more than m times and the **second** more than n times.
 - **Asymptotic computing time** of this algorithm is $O(n + m)$.
 - *One* of the **problems** with the current representation is that, while creating polynomials, *avail* is **incremented until** it equals **MAX-TERMS**.
 - When this occurs, must we quit?
 - A **compaction** function is written which would **remove** the **unnecessary polynomials** and **create** a **large, continuous available space** at one end of the array.
 - However, this requires **data movement** which takes **time**.
 - In addition, we also must **change** the **start** and **end indices** for each **polynomial moved** (i.e) the values of *starti* and *finishi* is *changed* for all polynomials moved.

SPARSE MATRIX

- A matrix is a mathematical object which arises in many physical problems.
- We are interested in studying ways to represent matrices so that the operations to be performed on them can be carried out efficiently.
- In general, we write $m \times n$ (read m by n) to designate a matrix with m rows and n columns. Such a matrix has mn elements.
- When m is equal to n , we call the matrix *square*.
- It is very natural to store a matrix in a two dimensional array, say $A(1:m, 1:n)$.
- Then we can work with any element by writing $A(i,j)$; and this element can be found very quickly
- A general matrix consists of m rows and n columns of numbers as in figure 2.1

Sparse Matrix

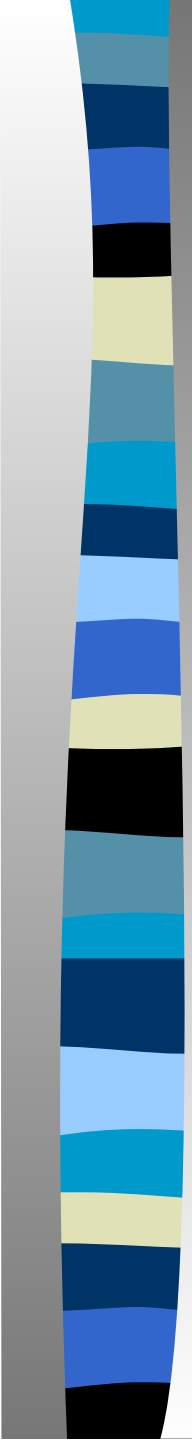
	col 1	col 2	col 3		col1	col2	col3	col4	col5	col6
row 1	-27	3	4	row0	15	0	0	22	0	-15
row 2	6	82	-2	row1	0	11	3	0	0	0
row 3	109	-64	11	row2	0	0	0	-6	0	0
row 4	12	8	9	row3	0	0	0	0	0	0
row 5	48	27	47	row4	91	0	0	0	0	0
				row5	0	0	28	0	0	0

(a) 15/15

(b) 8/36

***Figure 2.1: Two matrices**

↑
sparse matrix
data structure?

- 
- In fig2.1 b we see that it has many zero entries.
 - Such a matrix is called *sparse*.
 - There is no precise definition of when a matrix is sparse and when it is not, but it is a concept which we can all recognize intuitively.
 - Above, only 8 out of 36 possible elements are nonzero and that is sparse!
 - A sparse matrix requires us to consider an alternate form of representation.
 - This comes about because in practice many of the matrices we want to deal with are large, e.g., 1000 X 1000, but at the same time they are sparse: say only 1000 out of one million possible elements are nonzero.
 - On most computers today it would be impossible to store a full 1000 X 1000 matrix in the memory at once.
 - Therefore, we ask for an alternative representation for sparse matrices.
 - The alternative representation will explicitly store only the nonzero elements.

- Each element of a matrix is uniquely characterized by its row and column position, say i,j .
- We might then store a matrix as a list of 3-tuples of the form $(i,j,value)$.
- Also it might be helpful to organize this list of 3-tuples in some way, perhaps placing them so that the row numbers are increasing.
- We can go one step farther and require that all the 3-tuples of any row be stored so that the columns are increasing.
- Thus, we might store the second matrix of figure 2.1 in the array $A(0:t,1:3)$ where $t = 8$ is the number of nonzero terms.
- The elements $A(0,1)$ and $A(0,2)$ contain the number of rows and columns of the matrix. $A(0,3)$ contains the number of nonzero terms.

	1,	2,	3
A(0,	6,	6,	8
(1,	1,	1,	15
(2,	1,	4,	22
(3,	1,	6,	—15
(4,	2,	2,	11
(5,	2,	3,	3
(6,	3,	4,	—6
(7,	5,	1,	91
(8,	6,	3,	28

Figure 2.2: Sparse matrix stored as triples

SPARSE MATRIX ABSTRACT DATA TYPE

Structure *Sparse_Matrix* is

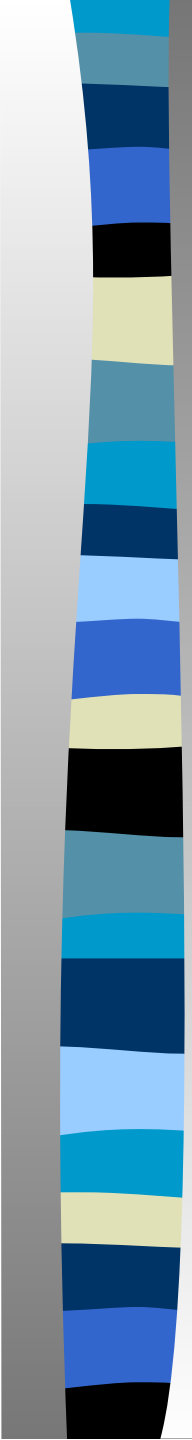
objects: a set of triples, $\langle row, column, value \rangle$, where *row* and *column* are integers and form a unique combination, and *value* comes from the set *item*.

functions:

for all $a, b \in \text{Sparse_Matrix}$, $x \in \text{item}$, i, j , max_col ,
 $max_row \in \text{index}$

Sparse_Marix **Create**(max_row, max_col) ::=

return a *Sparse_matrix* that can hold up to
 $max_items = max_row \times max_col$ and
whose maximum row size is max_row and
whose maximum column size is max_col .



Sparse_Matrix **Transpose**(a) ::=

return the matrix produced by interchanging the row and column value of every triple.

Sparse_Matrix **Add**(a, b) ::=

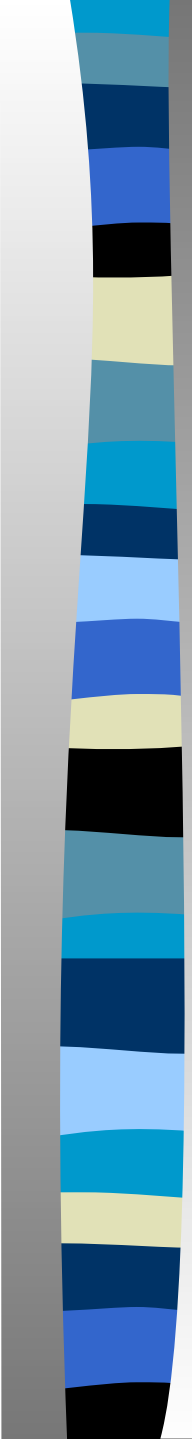
if the dimensions of a and b are the same
return the matrix produced by adding corresponding items, namely those with identical *row* and *column* values.
else return error

Sparse_Matrix **Multiply**(a, b) ::=

if number of columns in a equals number of rows in b
return the matrix d produced by multiplying a by b according to the formula: $d[i][j] = \sum (a[i][k] \cdot b[k][j])$ where $d(i, j)$ is the (i, j) th element
else return error.

- Now what are some of the operations we might want to perform on these matrices?
- One operation is to compute the transpose matrix.
- This is where we move the elements so that the element in the i,j position gets put in the j,i position.
- Another way of saying this is that we are interchanging rows and columns.
- The elements on the diagonal will remain unchanged, since $i = j$.
- The transpose of the example matrix looks like

	1,	2,	3
B(0,	6,	6,	8
(1,	1,	1,	15
(2,	1,	5,	91
(3,	2,	2,	11
(4,	3,	2,	3
(5,	3,	6,	28
(6,	4,	1,	22
(7,	4,	3,	-6
(8,	6,	1,	-15

- 
- Since A is organized by row, our first idea for a transpose algorithm might be

for each row i do
 take element (i,j,val) and
 store it in (j,i,val) of the transpose
end

- The difficulty is in not knowing where to put the element (j,i,val) until all other elements which precede it have been processed.
- In our example of figure 2.2, for instance, we have item
- $(1,1,15)$ which becomes $(1,1,15)$
- $(1,4,22)$ which becomes $(4,1,22)$
- $(1,6, - 15)$ which becomes $(6,1, - 15)$.
- If we just place them consecutively, then we will need to insert many new triples, forcing us to move elements down very often.
- We can avoid this data movement by finding the elements in the order we want them, which would be as

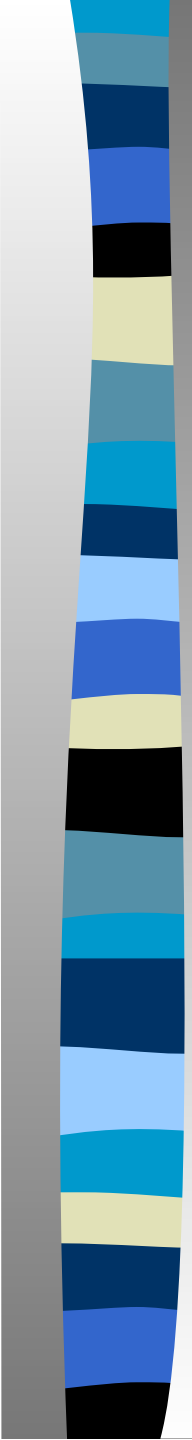
for all elements in column j do
 place element (i,j,val) in position (j,i,val)
end

- This says find all elements in column 1 and store them into row 1, find all elements in column 2 and Store them in row 2, etc.
- Since the rows are originally in order, this means that we will locate elements in the correct column order as well.
- Let us write out the algorithm in full.

```

procedure TRANSPOSE (A,B)
    // A is a matrix represented in sparse form //
    // B is set to be its transpose //
1   (m,n,t) ← (A(0,1),A(0,2),A(0,3))
2   (B(0,1),B(0,2),B(0,3)) ← (n,m,t)
3   if t ≤ 0 then return           // check for zero matrix //
4   q ← 1           // q is position of next term in B //
5   for col ← 1 to n do           // transpose by columns //
6       for p ← 1 to t do         // for all nonzero terms do //
7           if A(p,2) = col       // correct column //
8               then [(B(q,1),B(q,2),B(q,3)) ←           // insert next term
                                                                of B //
9                           (A(p,2),A(p,1),A(p,3))
10                          q ← q + 1]
11   end
12   end
13 end TRANSPOSE
  
```


- EXTRA SLIDES



Sparse_matrix Create(max_row, max_col) ::=

```
#define MAX_TERMS 101 /* maximum number of terms +1 */  
typedef struct {  
    int col;  
    int row;  
    int value;  
} term;  
term a[MAX_TERMS]
```

↑
of rows (columns)
of nonzero terms

*(P.69)

Transpose a Matrix

(1) for each **row** i

take element $\langle i, j, \text{value} \rangle$ and store it
in element $\langle j, i, \text{value} \rangle$ of the transpose.

difficulty: where to put $\langle j, i, \text{value} \rangle$

$(0, 0, 15) \implies (0, 0, 15)$

$(0, 3, 22) \implies (3, 0, 22)$

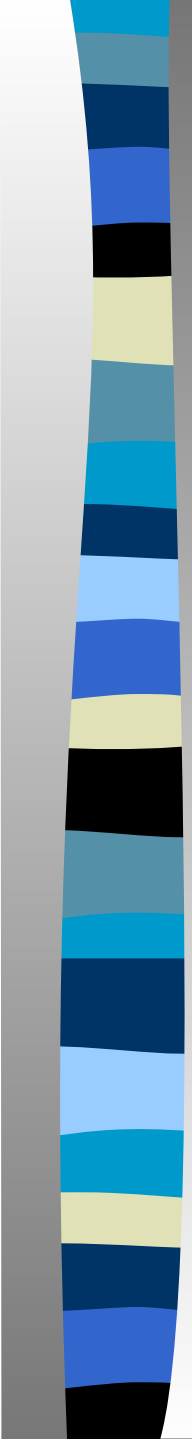
$(0, 5, -15) \implies (5, 0, -15)$

$(1, 1, 11) \implies (1, 1, 11)$

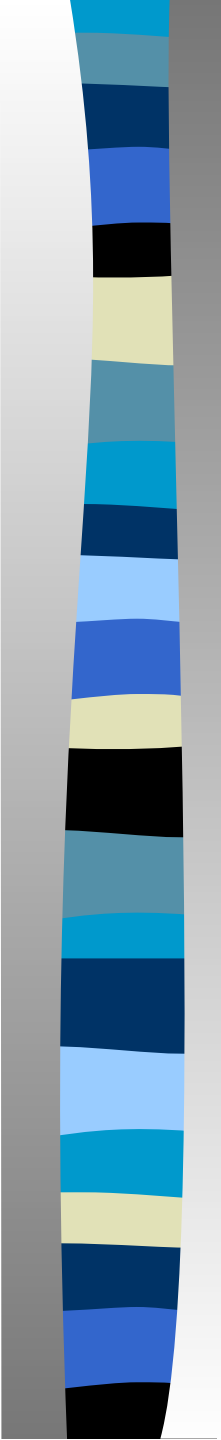
Move elements down very often.

(2) For all elements in **column** j ,

place element $\langle i, j, \text{value} \rangle$ in element $\langle j, i, \text{value} \rangle$



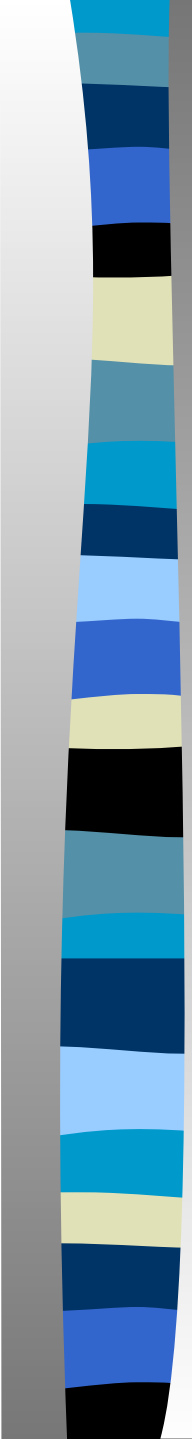
```
void transpose (term a[], term b[])
/* b is set to the transpose of a */
{
    int n, i, j, currentb;
    n = a[0].value; /* total number of elements */
    b[0].row = a[0].col; /* rows in b = columns in a */
    b[0].col = a[0].row; /* columns in b = rows in a */
    b[0].value = n;
    if (n > 0) { /*non zero matrix */
        currentb = 1;
        for (i = 0; i < a[0].col; i++)
            /* transpose by columns in a */
            for( j = 1; j <= n; j++)
                /* find elements from the current column */
                if (a[j].col == i) {
                    /* element is in current column, add it to b */
```



```
columns
{
    elements
    {
        b[currentb].row = a[j].col;
        b[currentb].col = a[j].row;
        b[currentb].value = a[j].value;
        currentb++;
    }
}
```

*** Program 2.7: Transpose of a sparse matrix (p.71)**

Scan the array “columns” times. $\implies O(\text{columns} * \text{elements})$
The array has “elements” elements.



Discussion: compared with 2-D array representation

$O(\text{columns} * \text{elements})$ vs. $O(\text{columns} * \text{rows})$

elements $\rightarrow$ columns * rows when nonsparse

$O(\text{columns} * \text{columns} * \text{rows})$

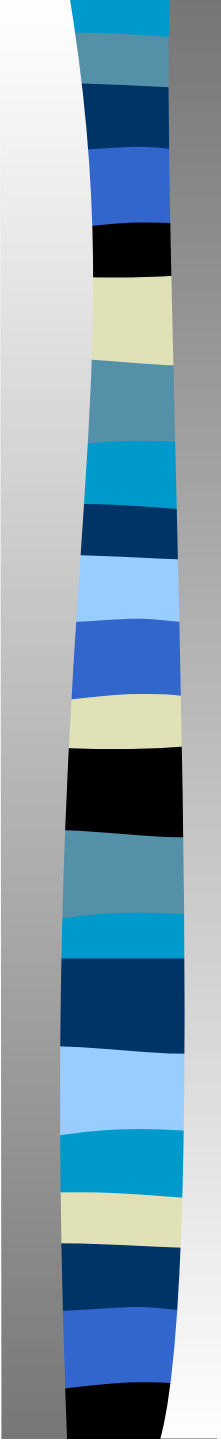
Problem: Scan the array “columns” times.

Solution:FT

Determine the number of elements in each column of the original matrix.

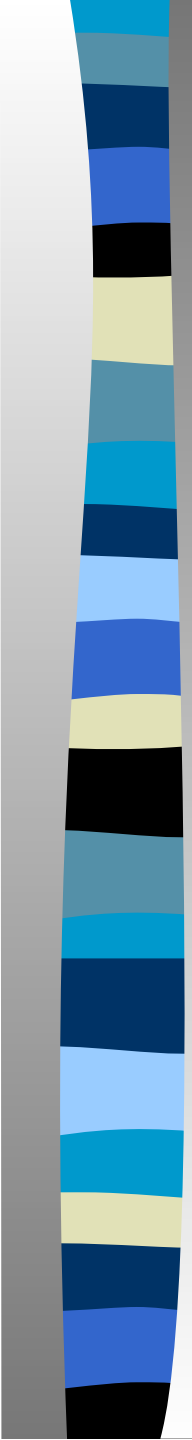
$\Rightarrow$

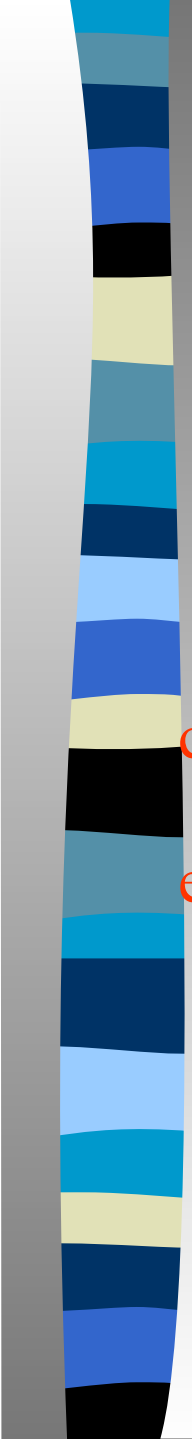
Determine the starting positions of each row in the transpose matrix.



a[0]	6	6	8
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28

	[0]	[1]	[2]	[3]	[4]	[5]
row_terms =	2	1	2	2	0	1
starting_pos =	1	3	4	6	8	8

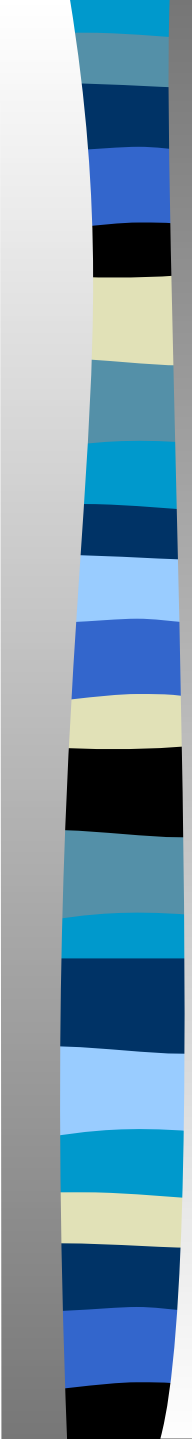
- 
- The $O\{columns \cdot elements\}$ time for our transpose function becomes $O(columns.columns.elements)$ when the number of elements is of the order $columns.rows$.
 - Perhaps, to **conserve space**, we have traded away **too much time**.
 - Actually, we can create a much better algorithm by using a **little more storage**.
 - In fact, we can transpose a matrix represented as a sequence of triples in $O\{columns + elements\}$ time.
 - This algorithm, *fast-transpose* (Program 2.8), proceeds by **first determining the number of elements in each column of the original matrix**.
 - This gives us the number of elements in each row of the transpose matrix.
 - From this information, we can determine **the starting position of each row in the transpose matrix**.
 - We now can move the elements in the original matrix one by one into their correct position in the transpose matrix.



```

void fast_transpose(term a[ ], term b[ ])
{
/* the transpose of a is placed in b */
int row_terms[MAX_COL], starting_pos[MAX_COL];
int i, j, num_cols = a[0].col, num_terms = a[0].value;
b[0].row = num_cols; b[0].col = a[0].row;
b[0].value = num_terms;
if (num_terms > 0){ /*nonzero matrix*/
    columns [for (i = 0; i < num_cols; i++)
              row_terms[i] = 0;
    elements [for (i = 1; i <= num_terms; i++)
              row_term [a[i].col]++
              starting_pos[0] = 1;
    columns [for (i = 1; i < num_cols; i++)
              starting_pos[i]=starting_pos[i-1] +row_terms [i-1];

```



elements

```
for (i=1; i <= num_terms, i++) {  
    j = starting_pos[a[i].col]++;  
    b[j].row = a[i].col;  
    b[j].col = a[i].row;  
    b[j].value = a[i].value;  
}  
}
```

*Program 2.8: Fast transpose of a sparse matrix

Compared with 2-D array representation

$O(\text{columns} + \text{elements})$ vs. $O(\text{columns} * \text{rows})$

elements $\rightarrow$ columns * rows

$O(\text{columns} + \text{elements}) \rightarrow O(\text{columns} * \text{rows})$

Cost: Additional `row_terms` and `starting_pos` arrays are required.

Let the two arrays `row_terms` and `starting_pos` be shared.

Sparse Matrix Multiplication

Definition: $[D]_{m \times p} = [A]_{m \times n} * [B]_{n \times p}$

Procedure: Fix a row of A and find all elements in column j of B for $j=0, 1, \dots, p-1$.

Alternative 1. Scan all of B to find all elements in j.

Alternative 2. Compute the transpose of B.

(Put all column elements consecutively)

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

An Example

$$A = \begin{bmatrix} 1 & 0 & 2 \\ -1 & 4 & 6 \end{bmatrix} \quad B = \begin{bmatrix} 3 & 0 & 2 \\ -1 & 0 & 0 \\ 0 & 0 & 5 \end{bmatrix}$$

$$\begin{array}{ccc|ccc} 2 & 3 & 5 & 3 & 3 & 4 \\ 0 & 0 & 1 & 0 & 0 & 3 \\ 0 & 2 & 2 & 0 & 2 & 2 \\ \hline 1 & 0 & -1 & 1 & 0 & -1 \\ 1 & 1 & 4 & 2 & 2 & 5 \\ 1 & 2 & 6 & & & \end{array}$$

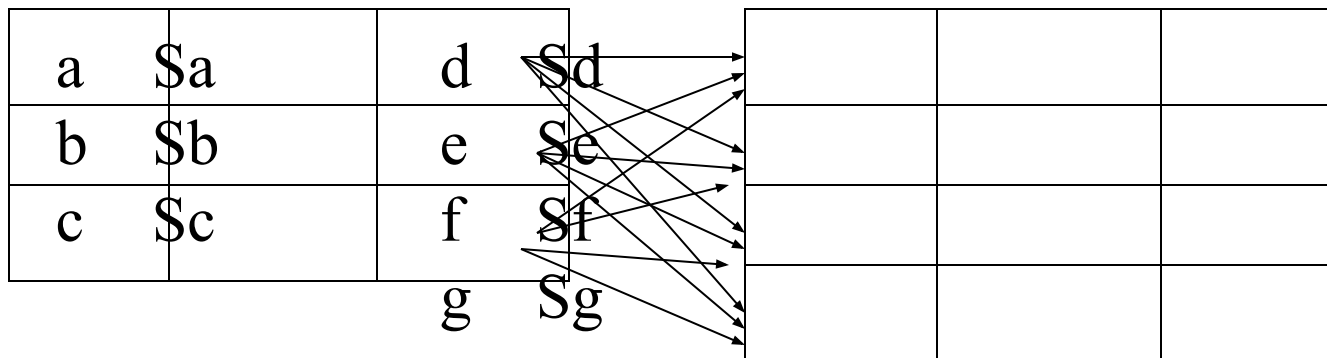
$$B^T = \begin{bmatrix} 3 & -1 & 0 \\ 0 & 0 & 0 \\ 2 & 0 & 5 \end{bmatrix} \quad \begin{array}{ccc|ccc} 3 & 3 & 3 & 3 & 3 & 3 \\ 0 & 0 & 3 & 0 & 0 & 3 \\ 0 & 1 & -1 & 0 & 1 & -1 \\ 2 & 0 & 2 & 2 & 0 & 2 \\ 2 & 2 & 5 & 2 & 2 & 5 \end{array}$$

$(0,0)$
 $(0,2)$
 $(1,0)$
 $(1,2)$

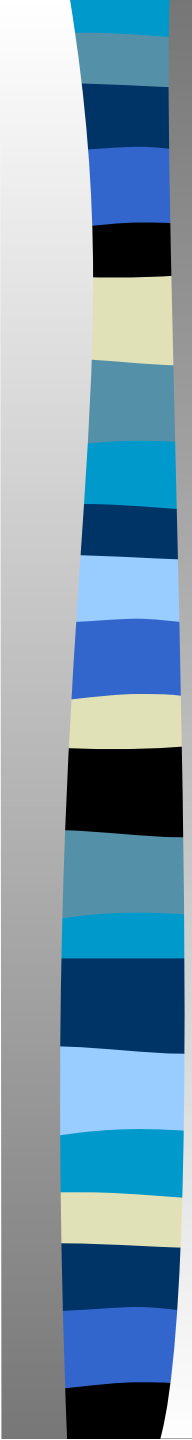
General Case

$$d_{ij} = a_{i0} * b_{0j} + a_{i1} * b_{1j} + \dots + a_{i(n-1)} * b_{(n-1)j}$$

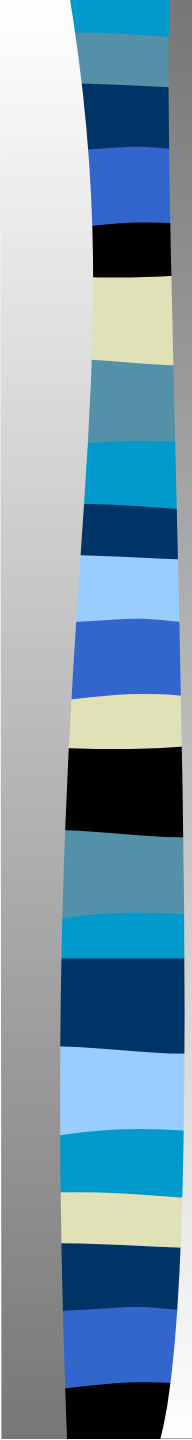
a本來依i成群，經轉置後，b也依j成群。



最多可以產生ad, ae, af, ag,
bd, be, bf, bg,
cd, ce, cf, cg 等entries。



```
void mmult (term a[ ], term b[ ], term d[ ] )
/* multiply two sparse matrices */
{
    int i, j, column, totalb = b[0].value, totald = 0;
    int rows_a = a[0].row, cols_a = a[0].col,
    totala = a[0].value; int cols_b = b[0].col,
    int row_begin = 1, row = a[1].row, sum = 0;
    int new_b[MAX_TERMS][3];
    if (cols_a != b[0].row){
        fprintf (stderr, "Incompatible matrices\n");
        exit (1);
    }
}
```



```
fast_transpose(b, new_b);  
/* set boundary condition */  
a[totala+1].row = rows_a;  
new_b[totalb+1].row = cols_b;  
new_b[totalb+1].col = 0;
```

$\text{cols_b} + \text{totalb}$

```
for (i = 1; i <= totala; ) {
```

at most rows_a times

```
    column = new_b[1].row;
```

```
    for (j = 1; j <= totalb+1;) {
```

```
        /* mutiply row of a by column of b */
```

```
        if (a[i].row != row) {
```

```
            storesum(d, &totald, row, column, &sum);
```

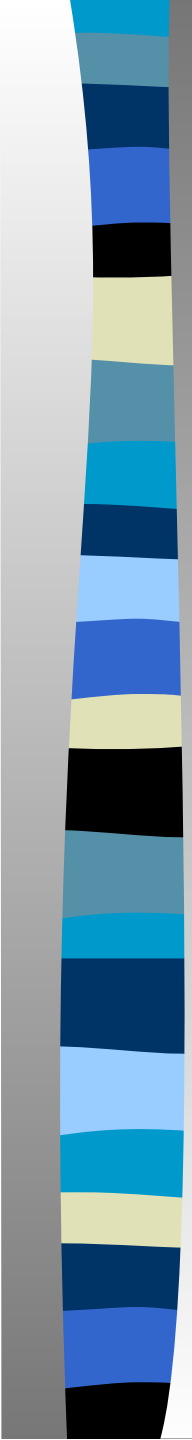
```
            i = row_begin;
```

```
            for (; new_b[j].row == column; j++)
```

```
                ;
```

```
            column = new_b[j].row
```

```
        }
```

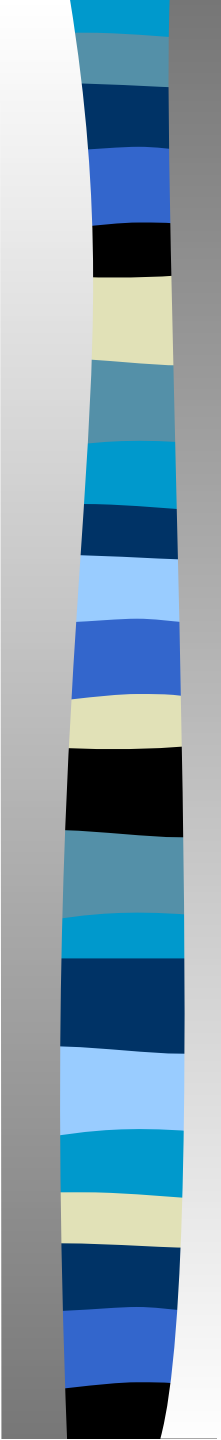


```

else switch (COMPARE (a[i].col, new_b[j].col)) {
    case -1: /* go to next term in a */
        i++; break;
    case 0: /* add terms, go to next term in a and b */
        sum += (a[i++].value * new_b[j++].value);
        break;
    case 1: /* advance to next term in b*/
        j++
    }
} /* end of for j <= totalb+1 */
for (; a[i].row == row; i++)
    ;
    row_begin = i; row = a[i].row;
} /* end of for i <=totala */
d[0].row = rows_a;
d[0].col = cols_b; d[0].value = totald;
}

```

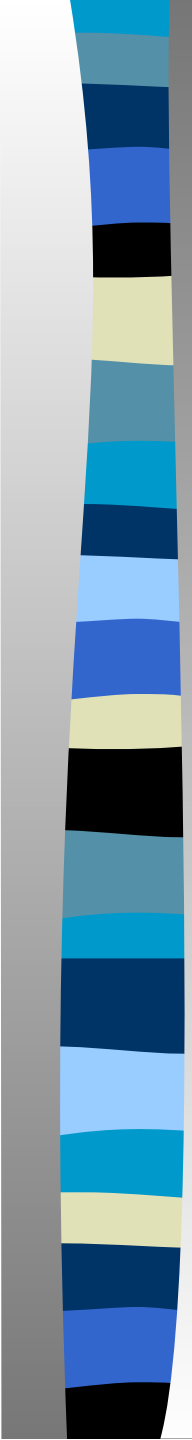
***Praogram 2.9: Sparse matrix multiplication (p.75)**



Analyzing the algorithm

$$\begin{aligned} & \text{cols_b} * \text{termsrow}_1 + \text{totalb} + \\ & \text{cols_b} * \text{termsrow}_2 + \text{totalb} + \\ & \dots + \\ & \text{cols_b} * \text{termsrow}_p + \text{totalb} \\ &= \text{cols_b} * (\text{termsrow}_1 + \text{termsrow}_2 + \dots + \text{termsrow}_p) + \\ & \quad \text{rows_a} * \text{totalb} \\ &= \text{cols_b} * \text{totala} + \text{row_a} * \text{totalb} \end{aligned}$$

$$O(\text{cols_b} * \text{totala} + \text{rows_a} * \text{totalb})$$



Compared with matrix multiplication using array

```
for (i=0; i < rows_a; i++)  
    for (j=0; j < cols_b; j++) {  
        sum =0;  
        for (k=0; k < cols_a; k++)  
            sum += (a[i][k] *b[k][j]);  
        d[i][j] =sum;  
    }
```

$O(\text{rows_a} * \text{cols_a} * \text{cols_b})$ vs.

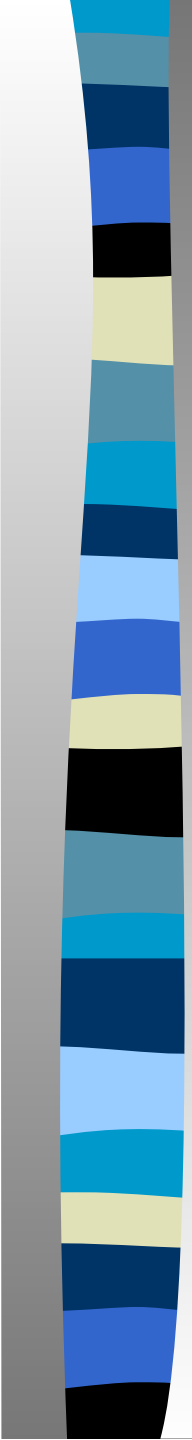
$O(\text{cols_b} * \text{total_a} + \text{rows_a} * \text{total_b})$

optimal case: $\text{total_a} < \text{rows_a} * \text{cols_a}$

$\text{total_b} < \text{cols_a} * \text{cols_b}$

worse case: $\text{total_a} \rightarrow \text{rows_a} * \text{cols_a}$, or

$\text{total_b} \rightarrow \text{cols_a} * \text{cols_b}$



```
void storesum(term d[ ], int *totald, int row, int column,
              int *sum)
{
/* if *sum != 0, then it along with its row and column
   position is stored as the *totald+1 entry in d */
   if (*sum)
       if (*totald < MAX_TERMS) {
           d[++*totald].row = row;
           d[*totald].col = column;
           d[*totald].value = *sum;
       }
       else {
           fprintf(stderr, "Numbers of terms in product
                           exceed %d\n", MAX_TERMS);
           exit(1);
       }
}
```